GRAPH THEORY & ALGORITHM DESIGN

Tarjan's Algorithm for Finding Bridges (Critical Connections)

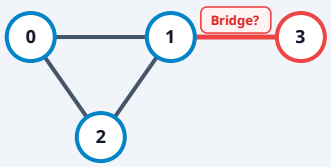
Complete Technical Guide with Subtree DFS Analysis & High-Contrast Vector Graphics

Problem Statement

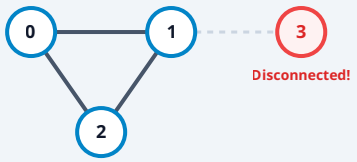
You are given an undirected connected graph representing a network of servers. A **critical connection (bridge)** is an edge whose removal disconnects the graph.

For example:

Original Server Network



After Removing Edge (1,3)



If we remove edge `(1,3)`, Node 3 becomes disconnected. Therefore, `(1,3)` is a **Bridge (Critical Connection)**.

Naive Approach

For every edge: (1) Remove the edge, (2) Run DFS/BFS, (3) Check if graph is disconnected.

> **Time Complexity:** For every edge → DFS = $O(E \times (V + E))$
> For $V = 100{,}000$, $E = 100{,}000$, this becomes extremely slow. We need a linear-time solution.

Observation

Suppose DFS reaches u ——— v where $u = ext{parent}$ and $v = ext{child}$.

Now ask: *"Can v or any node inside v's subtree reach u or any ancestor of u without using edge (u,v)?"*

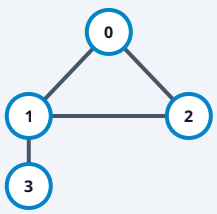
- If **YES** → removing (u,v) doesn't disconnect the graph.
- If **NO** → (u,v) is the only path. Hence it is a **Bridge**.

The whole algorithm is built around answering this question efficiently.

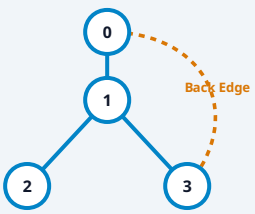
DFS Tree & Two Types of Edges

Suppose graph:

Original Graph



Generated DFS Tree



Here, `0 → 1`, `1 → 2`, `1 → 3` are **Tree Edges**. But original graph also contains `2 ---- 0` which is NOT part of DFS Tree. That edge is called a **Back Edge**.

Two Types of Edges

1. Tree Edge

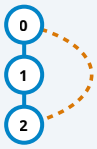
Edge used to visit unvisited node.

DFS: `0 → 1 → 2` | Edges: `0-1`, `1-2`

2. Back Edge

Edge to visited ancestor.

DFS: `0 → 1 → 2` | Back edge: `2 ---- 0`



Back edges create cycles. Cycles mean alternate paths. Alternate paths mean edges are not bridges.

Core Idea & The Two Tracking Arrays

We need to know: *Can this subtree reach an ancestor?* To answer this, Tarjan introduced two arrays.

Array 1 : `tin[]` (Time of Insertion / Entry Time)

`tin[node]` means: **At what time was this node first visited?**

Example DFS order `0 → 1 → 2 → 3`:

Node	0	1	2	3
tin[]	1	2	3	4

This value never changes. Think of it as **Entry Time**.

Why do we need tin[]? Suppose `0 — 1 — 2`. Without storing visit order, we don't know: *Which node is ancestor? Which node came first?* So we store `tin[]`.

Array 2 : `low[]` (Lowest Reachable Entry Time)

This is the **heart of Tarjan's Algorithm**.

> **Definition:** `low[node]` = The minimum discovery time (`tin`) reachable from this node or any node in its DFS subtree using **zero or one back edge**.

Initially `low[node] = tin[node]` because every node can reach itself. Later, it may become smaller.

Understanding `low[]` Step-by-Step

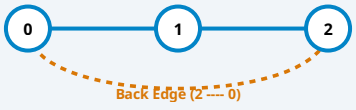
Example 1: Linear Path (No Back Edge)

Graph: `0 — 1 — 2` | DFS: `0 → 1 → 2`

Initially: `tin[0]=1, tin[1]=2, tin[2]=3` | `low[0]=1, low[1]=2, low[2]=3`. Nothing changes.

Example 2: Graph with a Back Edge

Graph with back edge `2 ---- 0`:



Step-by-Step Trace:

1. Initial: `tin`: `0=1, 1=2, 2=3` | `low`: `0=1, 1=2, 2=3`
2. Node 2 has back edge `2 ---- 0`: `low[2] = min(low[2], tin[0]) = min(3, 1) = 1`
3. Return to node 1: `low[1] = min(low[1], low[2]) = min(2, 1) = 1`
4. Return to node 0: `low[0] = min(low[0], low[1]) = min(1, 1) = 1`

Final State:

Node	0	1	2
tin[]	1	2	3
low[]	1	1	1

Interpretation: Although node 2 was discovered at time 3, it can actually reach node 0 whose discovery time is 1. Hence `low[2] = 1`.

Difference Between `tin[]` and `low[]`

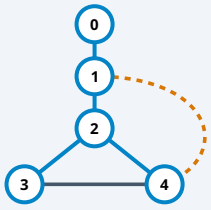
Notice: `tin` never changes once assigned, but `low` keeps decreasing as back edges are discovered during tree propagation.

Why Update `low[]` Differently?

Case 1: Child not visited (Tree Edge)
Run `DFS(child)`. After DFS returns:
`low[parent] = min(low[parent], low[child])`
Why? Child may have discovered a path to an ancestor. We propagate that upward.

Case 2: Already visited (Back Edge)
Update:
`low[node] = min(low[node], tin[ancestor])`
(**NOT** `low[ancestor]`)

Why use `tin[]` instead of `low[]` for Back Edge?



When node 4 has a back edge to node 1, we directly know it reaches node 1. We do not want to inherit all information stored inside `low[1]`, because `low[1]` may have been reduced due to completely different branches explored earlier.

A back edge tells us only that we can directly reach ancestor 1, so we use:

```
low[current] = min(low[current], tin[ancestor])
```

This keeps the low-link values correct.

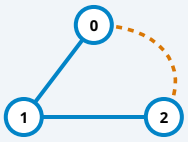
Bridge Condition

Suppose u ——— v where u = *ext{parent}* and v = *ext{child}*.

After `DFS(v)`, if: ***low[v] > tin[u]***
then the subtree rooted at v cannot reach u or any ancestor of u.
Therefore, edge ***(u,v)*** is the only connection. Hence, **Bridge**.

Why STRICTLY Greater (>)?

Example 1: Cycle Present



DFS: `0 → 1 → 2`, Back edge `2 → 0`
`low[2]=1, tin[1]=2`
Check `low[2] > tin[1]` → `1 > 2` (False)
NO BRIDGE

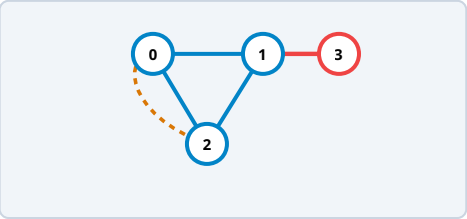
Example 2: No Cycle

No cycle: `tin[1]=2, tin[2]=3`
`low[2]=3, tin[1]=2`
Check `low[2] > tin[1]` → `3 > 2` (True)
BRIDGE: (1,2)

Complete Algorithm

1. **Step 1:** Build adjacency list.
2. **Step 2:** Initialize `timer = 1`, `tin = -1`, `low = -1`, `visited = false`.
3. **Step 3:** Run DFS.
4. **Step 4:** Visit node: `visited[node] = true`, `tin[node] = low[node] = timer++`.
5. **Step 5:** For every neighbour:
    6. If parent → `continue`.
    7. If unvisited: `DFS(child)`, `low[node] = min(low[node], low[child])`. Check: `if (low[child] > tin[node])` → **Bridge!**
    8. If already visited (Back Edge): `low[node] = min(low[node], tin[child])`.
9. **Step 6:** Return all bridges.

Dry Run



DFS Order: `0 ↓ → 1 ↓ → 2 ↓ → 3`

Initially: `tin` : `0=1, 1=2, 2=3, 3=4` | `low` : `0=1, 1=2, 2=3, 3=4`

Back edge `2 → 0` → Update `low[2] = 1`

Return to 1 → `low[1] = min(2, low[2]) = 1`

Node	0	1	2	3
tin	1	2	3	4
low	1	1	1	4

Checking Edges:

- Edge `1-3` : `low[3] = 4, tin[1] = 2` → `4 > 2` (TRUE) → **Bridge!**
- Edge `1-2` : `low[2] = 1, tin[1] = 2` → `1 > 2` (FALSE) → Not a bridge.

> **Final Answer:** Identified Bridge Edge = **(1, 3)**

Complexity Analysis

Metric	Complexity	Explanation
Time Complexity	$O(V + E)$	Building adjacency list: $O(V + E)$ | DFS traversal: $O(V + E)$. **Total:** $O(V + E)$
Space Complexity	$O(V + E)$	Adjacency list: $O(V + E)$ | `visited[]` , `tin[]` , `low[]` : $O(V)$ | DFS recursion stack: $O(V)$ worst case.

Key Points to Remember (Interview Cheat Sheet)

- `tin[node]` = Time when the node is first discovered during DFS. It never changes.
- `low[node]` = The smallest discovery time reachable from that node or its DFS subtree using zero or one back edge.
- For an unvisited child, update: `low[u] = min(low[u], low[v])` .
- For a back edge, update: `low[u] = min(low[u], tin[v])` .
- An edge (u, v) is a bridge if and only if `low[v] > tin[u]` .
- The algorithm works because `low[]` summarizes whether a subtree has an alternate path to any ancestor. If it doesn't, the parent-child edge is the only connection and is therefore critical.